



**BM CỤNG NGHỆ PHẦN MỀM — KHOA CNTT**  
**SE.NUCE.EDU.VN**

---

# **CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN NÂNG CAO**

## **Chương 5. Đồ thị**

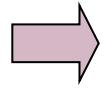
# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu

# Chương 5: Đồ thị



5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu

# 5.1. Đồ thị và các khái niệm liên quan

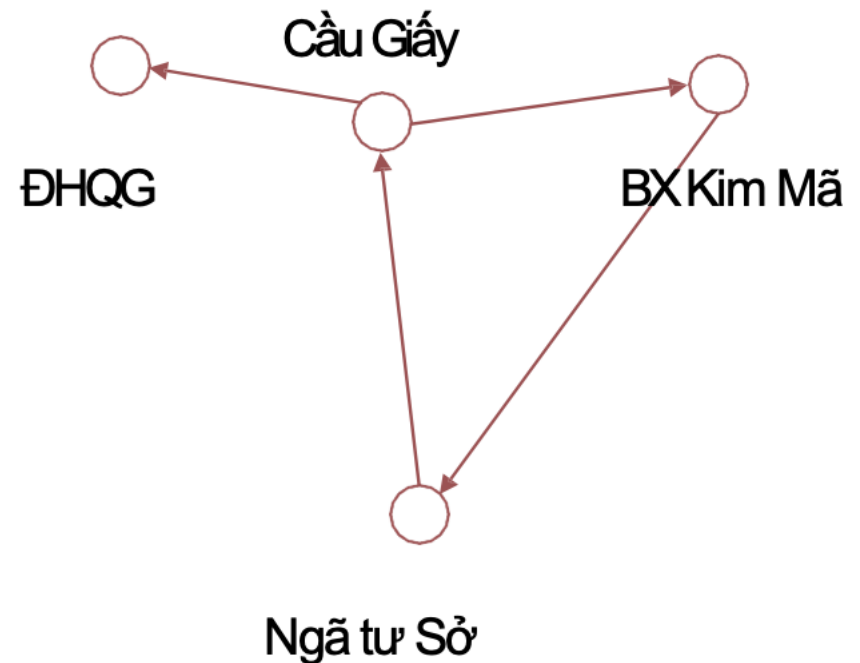
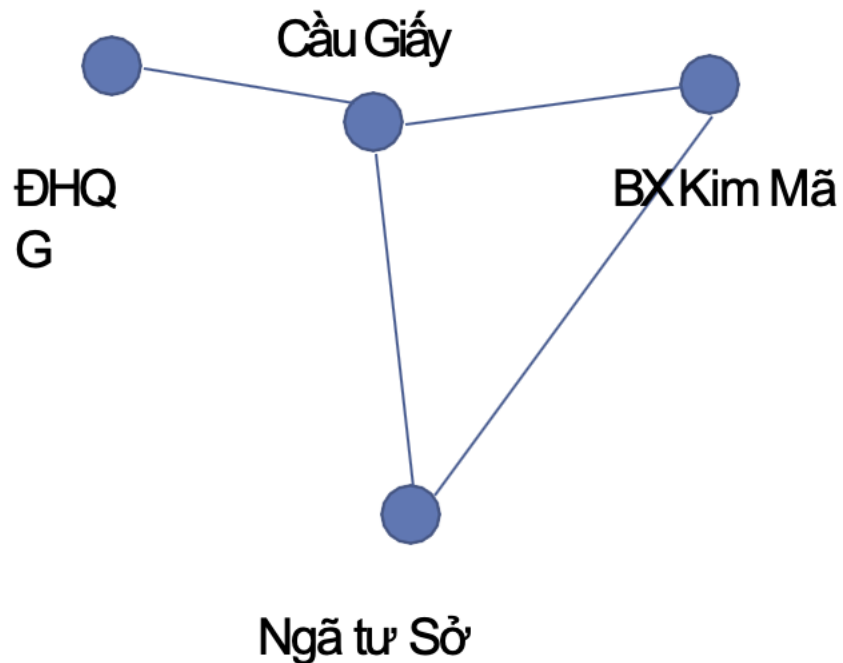
- Định nghĩa: Đồ thị là một mô hình toán học được sử dụng để biểu diễn một tập đối tượng có quan hệ với nhau theo một cách nào đó.
- Định nghĩa hình thức
  - Đồ thị  $G$  được xác định bởi một cặp  $(V, E)$ , trong đó
    - $V$  là tập đỉnh
    - $E$  là tập các cạnh nối cặp đỉnh  $E \subseteq \{(u, v) \mid u, v \in V\}$
- Đồ thị vô hướng
  - quan hệ định nghĩa bởi mỗi cạnh là quan hệ đối xứng
  - $E \subseteq \{\{u, v\} \mid u, v \in V\}$
- Đồ thị định hướng
  - $(u, v) \neq (v, u)$

# 5.1. Đồ thị và các khái niệm liên quan



## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ: đồ thị vô hướng – định hướng

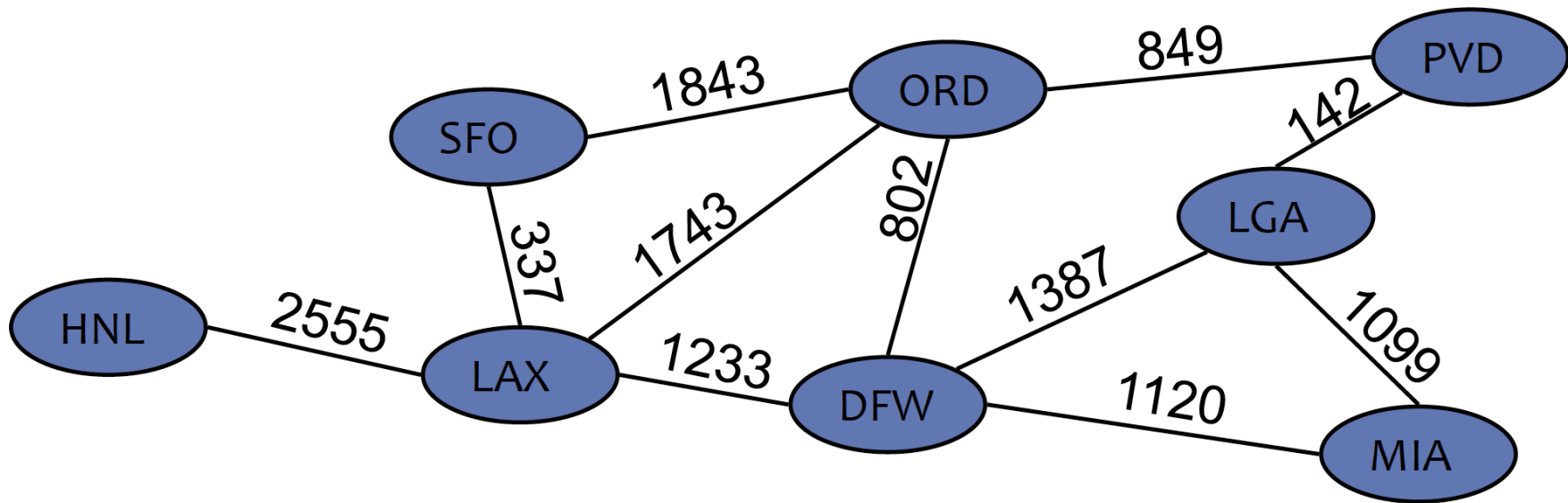


# 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ:
  - Mạng vận tải (transportation networks)
  - Mạng liên lạc (communication networks)
  - Mạng thông tin (information networks)
  - Mạng xã hội (social networks)
  - Mạng phụ thuộc (dependency networks)

## 5.1. Đồ thị và các khái niệm liên quan

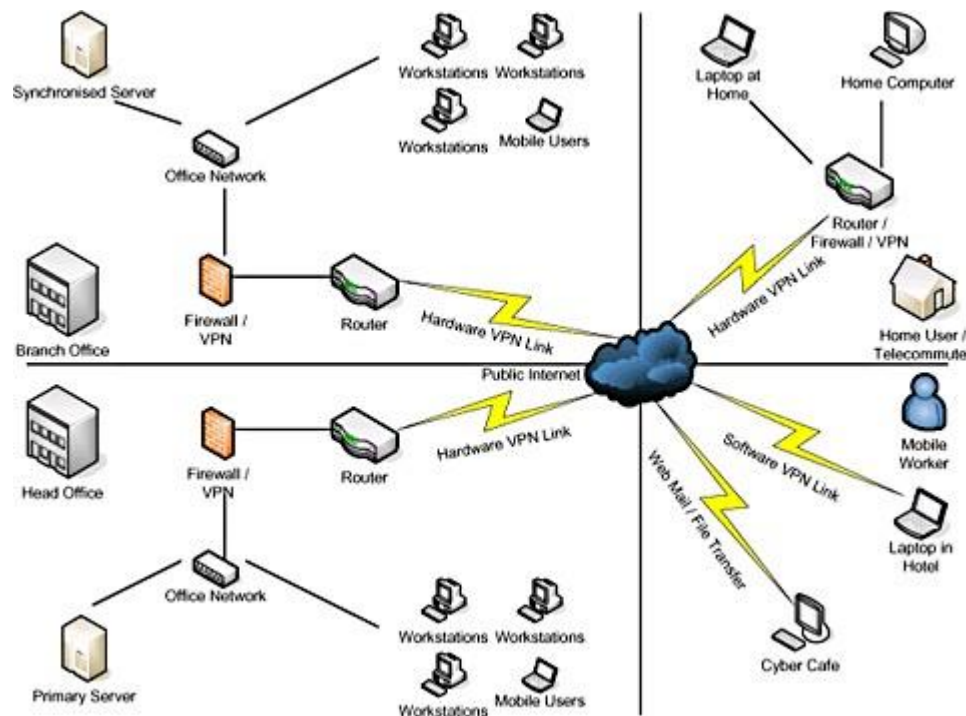
- Ví dụ:





# 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ:



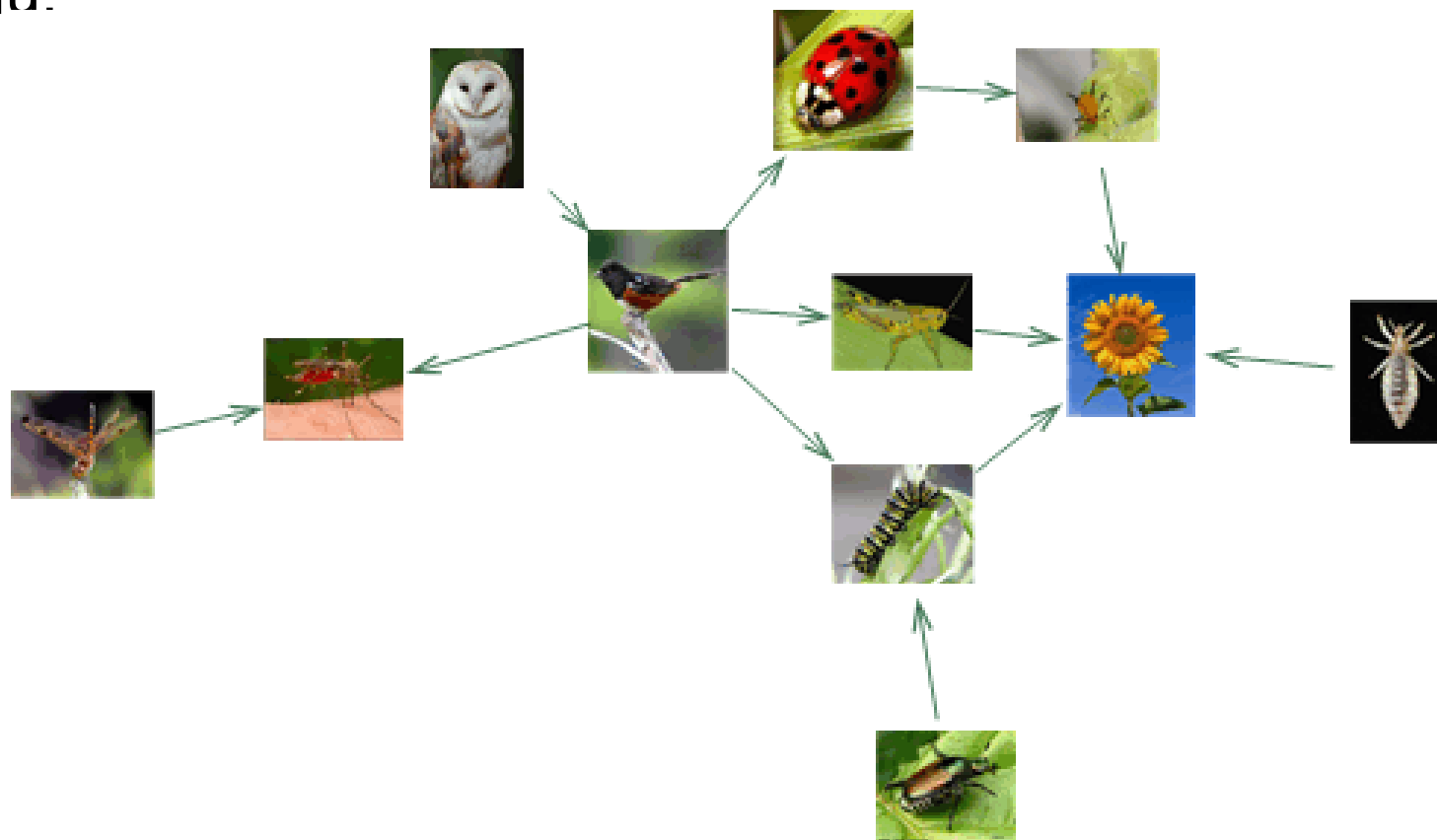
## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ:



## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ:

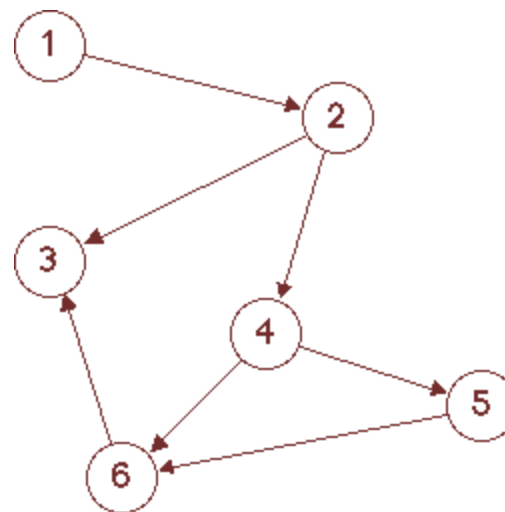
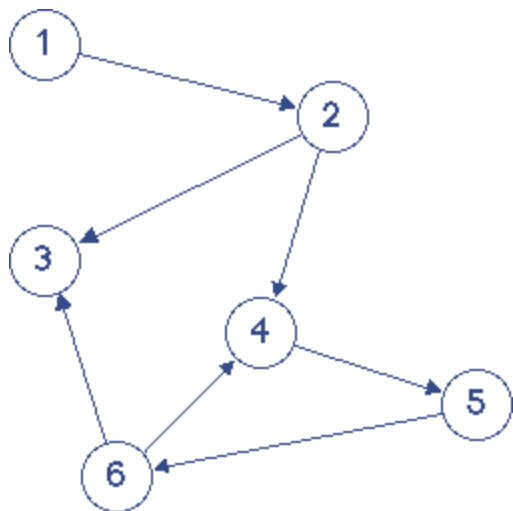


# 5.1. Đồ thị và các khái niệm liên quan

- Định nghĩa: Đường đi
  - Trong đồ thị vô hướng  $G=(V,E)$ 
    - Đường đi
      - là dãy  $P$  các đỉnh  $v_1, v_2, \dots, v_k$  có tính chất 2 đỉnh liên tiếp  $v_i, v_{i+1}$  được nối bởi 1 cạnh trong  $G$ .
      - $P$  được gọi là đường đi từ  $v_1$  đến  $v_k$
    - Chu trình là đường đi  $v_1, v_2, \dots, v_k$  với  $k > 2$  trong đó  $k-1$  đỉnh đầu tiên phân biệt và  $v_1 = v_k$
  - Với đồ thị có hướng, trong một đường đi hay chu trình, 2 đỉnh liên tiếp  $(v_i, v_{i+1})$  phải là một cung thuộc  $E$

## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ: đồ thị có chu trình – không có chu trình

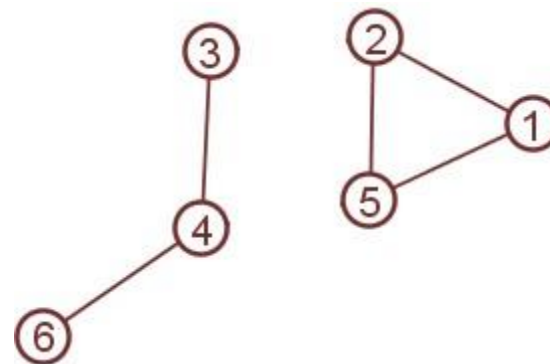
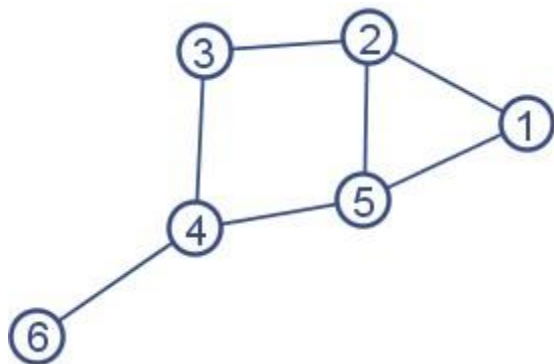


## 5.1. Đồ thị và các khái niệm liên quan

- Định nghĩa: Tính liên thông
  - Đồ thị vô hướng liên thông nếu tồn tại đường đi từ  $u$  đến  $v$  với mọi cặp đỉnh  $(u, v)$
  - Đồ thị có hướng
    - liên thông yếu nếu đồ thị vô hướng nền tảng của nó là đồ thị liên thông
    - liên thông mạnh nếu tồn tại một đường đi từ  $u$  đến  $v$  và một đường đi từ  $v$  đến  $u$  với mọi cặp đỉnh  $(u, v)$

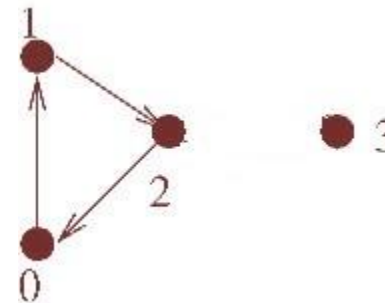
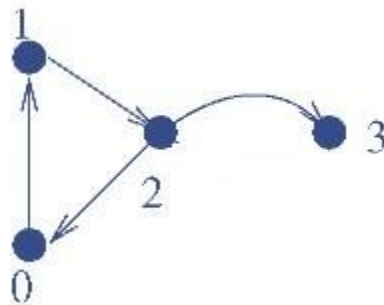
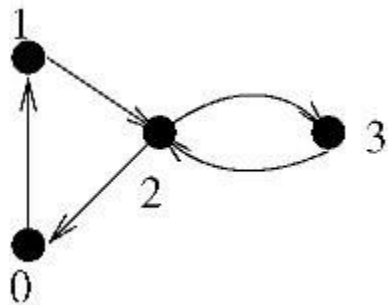
## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ: đồ thị vô hướng liên thông – không liên thông



## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ : đồ thị có hướng liên thông mạnh - yếu - không liên thông

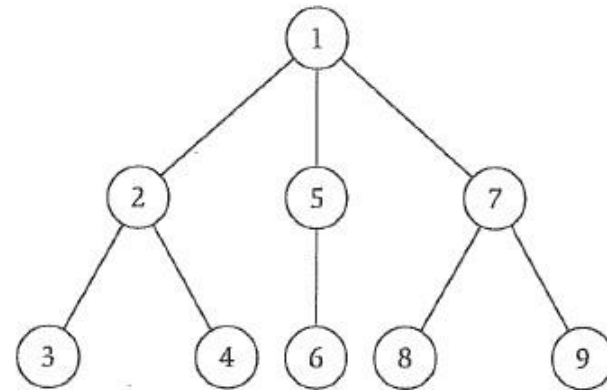
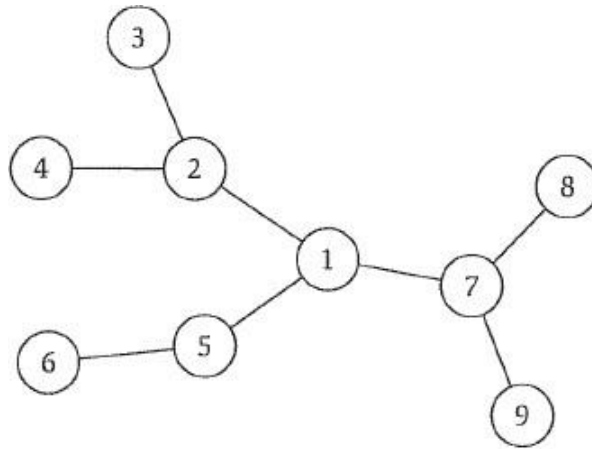




# 5.1. Đồ thị và các khái niệm liên quan

- Các khái niệm khác

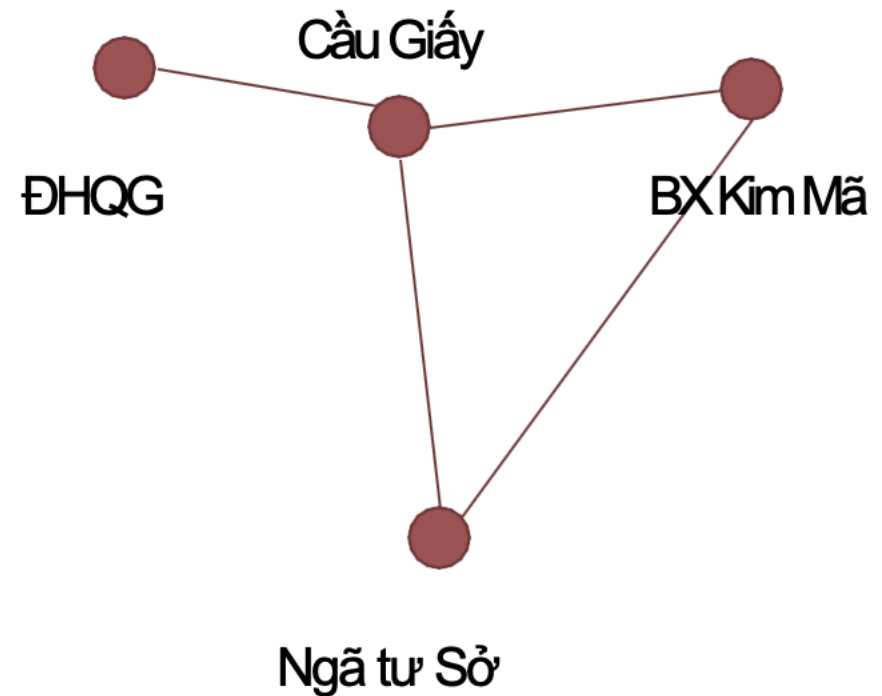
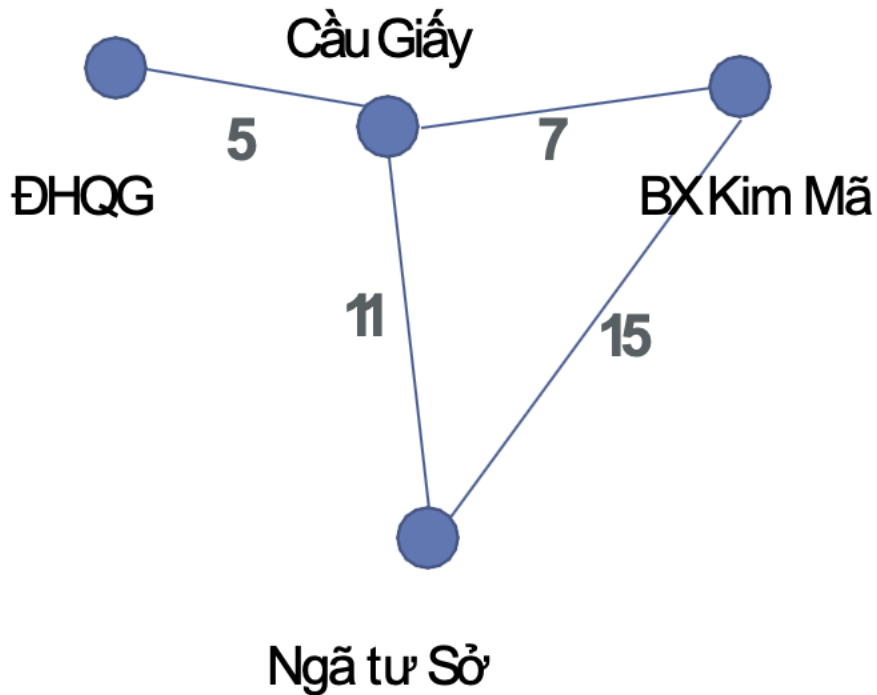
- Khoảng cách giữa 2 đỉnh  $u, v$  là số cạnh trên đường đi ngắn nhất từ  $u$  đến  $v$
- Cây trong lý thuyết đồ thị: là đồ thị vô hướng liên thông không chứa chu trình



- Đồ thị có/không có trọng số
- Đồ thị có/không có nhãn

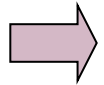
## 5.1. Đồ thị và các khái niệm liên quan

- Ví dụ: đồ thị có trọng số - không trọng số



# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

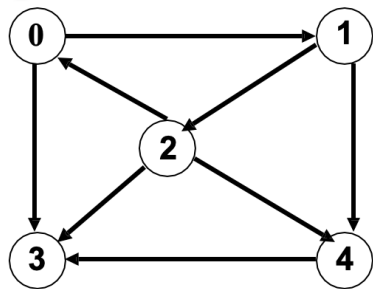


5.2. Cài đặt đồ thị

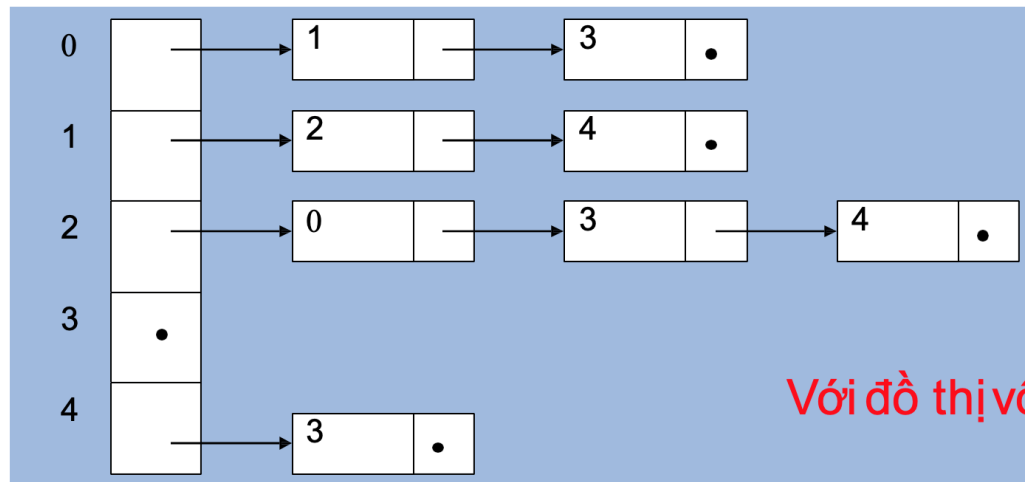
5.3. Một số bài toán tiêu biểu

## 5.2. Cài đặt đồ thị

- Hai cách biểu diễn đồ thị



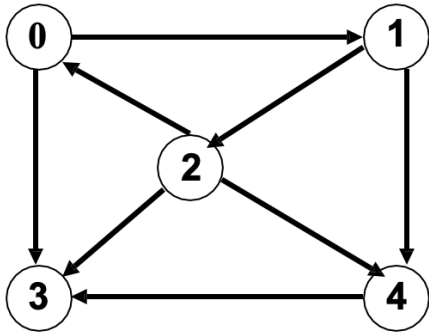
|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 2 | 1 | 0 | 0 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 1 | 0 |



Với đồ thị vô hướng?

## 5.2. Cài đặt đồ thị

- Cài đặt: Biểu diễn bằng ma trận kề

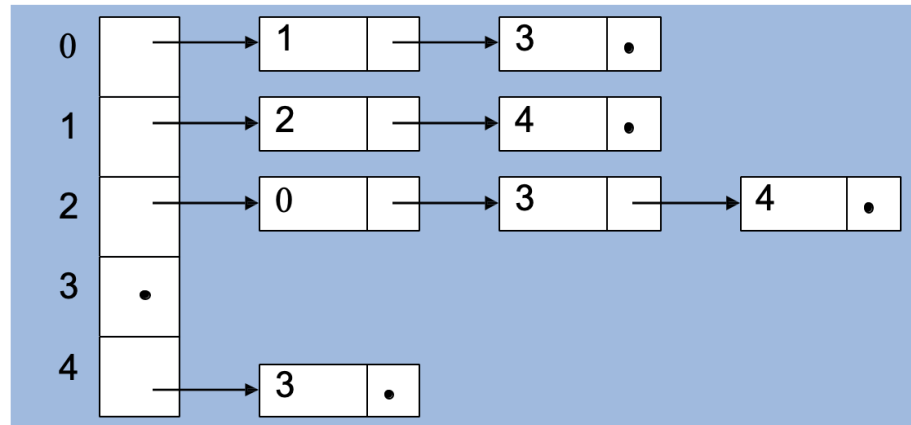
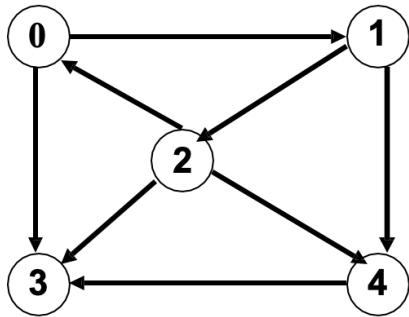


|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 2 | 1 | 0 | 0 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 1 | 0 |

```
const int N = 5;  
typedef bool Graph[N][N];  
...  
Graph g1;  
g1[0][0] = 0;  
g1[0][1] = 1;  
...
```

## 5.2. Cài đặt đồ thị

- Cài đặt: Biểu diễn bằng danh sách kề



```
struct Cell{
    int vertex;
    Cell * next;
};

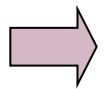
const int N = 5;
typedef Cell * Graph[N];

...
Graph g2;
addFirst(g2[0], 3);
addFirst(g2[0], 1);
```

# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị



5.3. Một số bài toán tiêu biểu

# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu



5.3.1. Đi qua đồ thị

5.3.2. Sắp xếp topo

5.3.3. Tìm đường đi ngắn nhất

5.3.4. Tìm cây bao trùm ngắn nhất

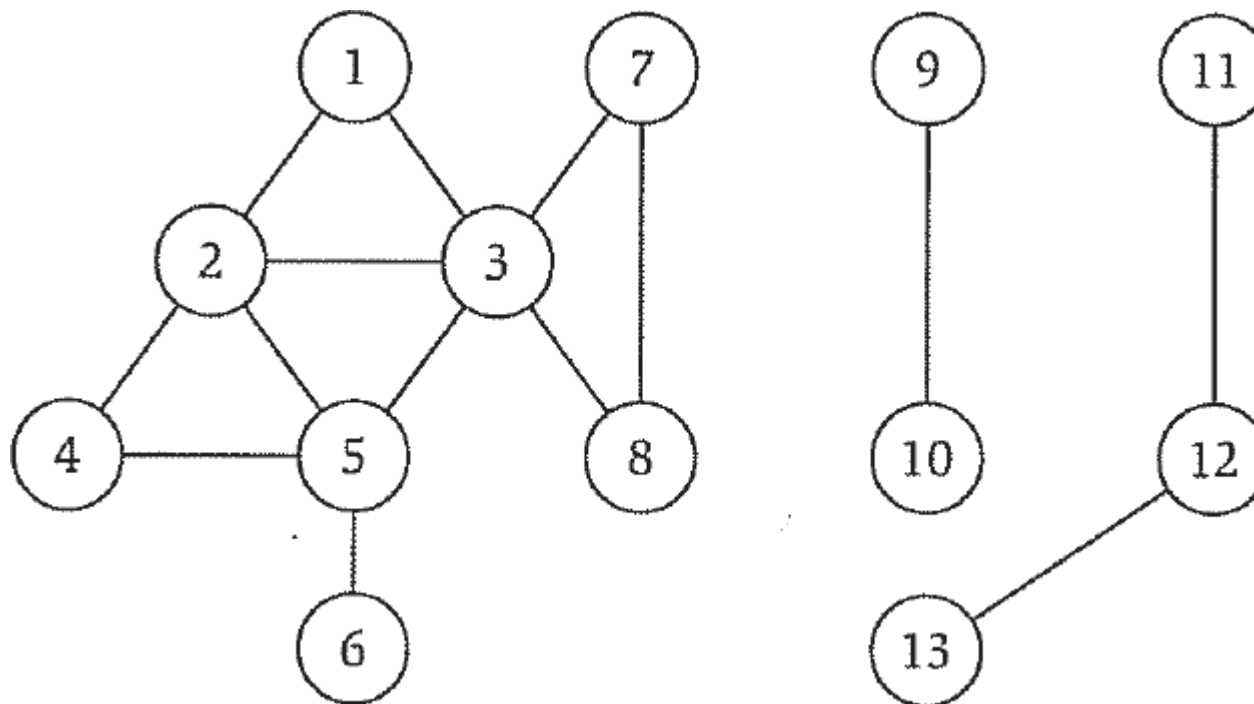


## 5.3. Một số bài toán tiêu biểu

- Đi qua đồ thị theo bề rộng
  - Sử dụng kĩ thuật tìm kiếm theo bề rộng
    - Breadth-First Search
- Ý tưởng của tìm kiếm theo bề rộng xuất phát từ đỉnh  $v$ 
  - Từ đỉnh  $v$  ta lần lượt đi thăm tất cả các đỉnh  $u$  kề đỉnh  $v$  mà  $u$  chưa được thăm.
  - Sau đó, đỉnh nào được thăm trước thì các đỉnh kề nó cũng sẽ được thăm trước.
  - Quá trình trên sẽ được tiếp tục cho tới khi ta không thể thăm đỉnh nào nữa.

## 5.3. Một số bài toán tiêu biểu

- Ví dụ



## 5.3. Một số bài toán tiêu biểu

- Ví dụ

Algorithm *BFS*(*v*)

// Tìm kiếm theo bề rộng xuất phát từ *v*.

*Input*: Đỉnh *v* chưa được thăm; Khởi tạo hàng đợi *Q* rỗng;

Đánh dấu đỉnh *v* đã được thăm;

*Q*.enqueue(*v*)

while *Q*.empty() ≠ TRUE

*w* ← *Q*.dequeue()

    for (mỗi đỉnh *u* kề *w*)

        if ( *u* chưa được thăm)

            Đánh dấu *u* đã được thăm;

*Q*.enqueue(*u*)

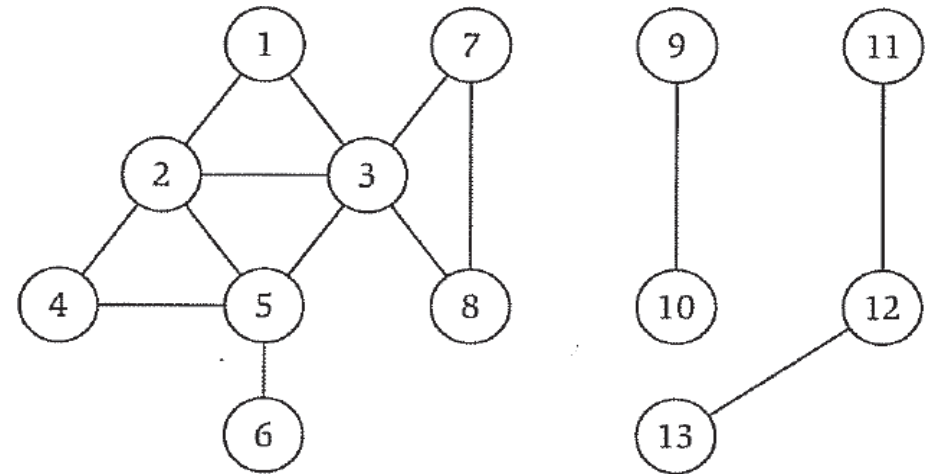
## 5.3. Một số bài toán tiêu biểu

Algorithm *BFS*Traversal( $G$ )

- // Đi qua đồ thị  $G=(V, E)$  theo bề rộng  
for (mỗi  $v \in V$ )  
Đánh dấu  $v$  chưa được thăm;  
for (mỗi  $v \in V$ )  
if ( $v$  chưa được thăm)  
BFS( $v$ );
- Ứng dụng
  - Vấn đề đạt tới: Giả sử  $v$  và  $w$  là hai đỉnh bất kỳ, ta muốn biết từ đỉnh  $v$  có đường đi tới đỉnh  $w$  hay không?
  - Tính liên thông và thành phần liên thông của đồ thị vô hướng

## 5.3. Một số bài toán tiêu biểu

| Bước | Đỉnh lấy ra | Visited sau bước này     | Queue sau bước này |
|------|-------------|--------------------------|--------------------|
| 0    | —           | {1}                      | [1]                |
| 1    | 1           | {1, 2, 3}                | [2, 3]             |
| 2    | 2           | {1, 2, 3, 4, 5}          | [3, 4, 5]          |
| 3    | 3           | {1, 2, 3, 4, 5, 7, 8}    | [4, 5, 7, 8]       |
| 4    | 4           | {1, 2, 3, 4, 5, 7, 8}    | [5, 7, 8]          |
| 5    | 5           | {1, 2, 3, 4, 5, 6, 7, 8} | [7, 8, 6]          |
| 6    | 7           | {1, 2, 3, 4, 5, 6, 7, 8} | [8, 6]             |
| 7    | 8           | {1, 2, 3, 4, 5, 6, 7, 8} | [6]                |
| 8    | 6           | {1, 2, 3, 4, 5, 6, 7, 8} | []                 |



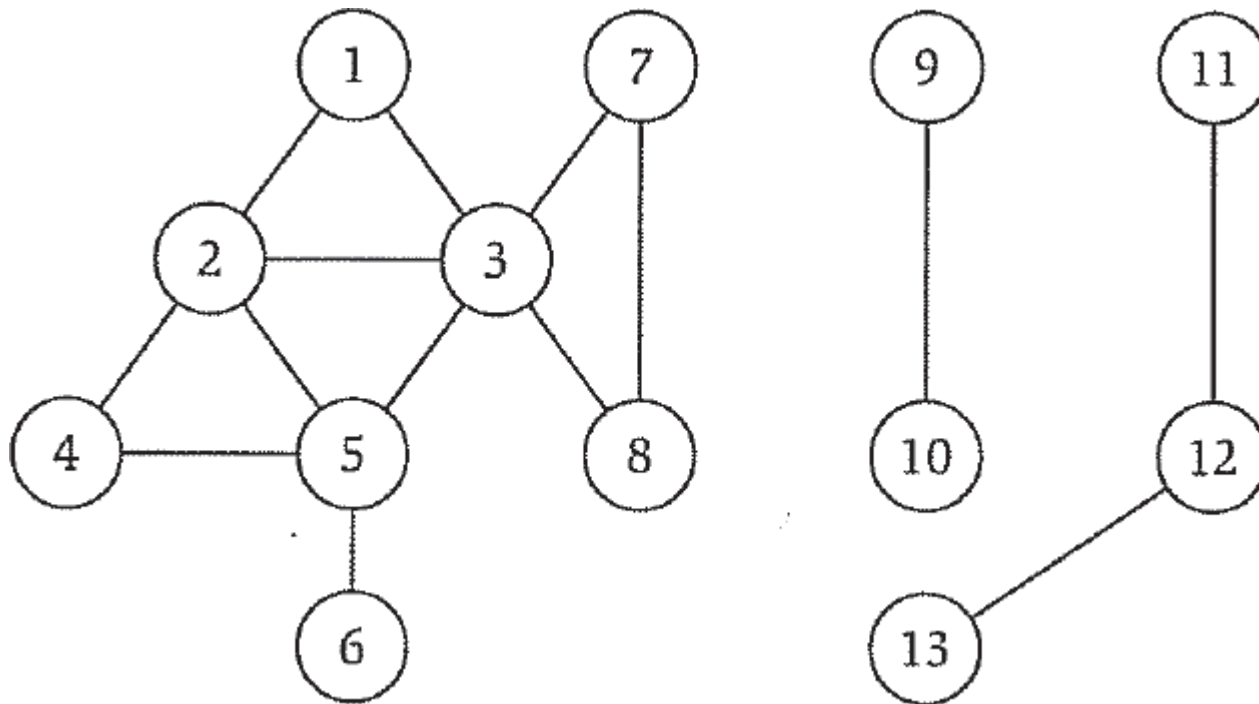
**Kết quả BFS**     $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 6$

## 5.3. Một số bài toán tiêu biểu

- Đi qua đồ thị theo độ sâu
  - Sử dụng kĩ thuật tìm kiếm theo độ sâu
    - **Depth-First Search**
  - Ý tưởng của tìm kiếm theo độ sâu xuất phát từ đỉnh  $u$ 
    - Từ đỉnh  $u$  ta đến thăm một đỉnh  $v$  kề đỉnh  $u$ . Rồi lại từ đỉnh  $v$  ta đến thăm đỉnh  $w$  kề  $v$ . Cứ thế tiếp tục chừng nào có thể được.
    - Khi đạt tới đỉnh  $v$  mà tại  $v$  ta không đi thăm tiếp được thì
      - Quay lại đỉnh  $u$  và từ đỉnh  $u$  ta đi thăm đỉnh  $v'$  khác kề  $u$  (nếu có), rồi từ  $v'$  lại đi thăm tiếp đỉnh kề  $v'$ ,...
      - Quá trình trên sẽ tiếp diễn cho tới khi ta không thể tới thăm đỉnh nào nữa.

## 5.3. Một số bài toán tiêu biểu

- Ví dụ DFS



## 5.3. Một số bài toán tiêu biểu

Algorithm *DFS*( $v$ )

// Tìm kiếm theo độ sâu xuất phát từ  $v$ .

*Input*: Đỉnh  $v$  chưa được thăm

for (mỗi đỉnh  $u$  kề  $v$ )

if (  $u$  chưa được thăm)

Đánh dấu  $u$  đã được thăm; *DFS*( $u$ )



## 5.3. Một số bài toán tiêu biểu

- Thuật toán đi qua đồ thị  $G$  theo độ sâu

Algorithm *DFSTraversal*( $G$ )

// Đi qua đồ thị  $G=(V, E)$  theo độ sâu

for (mỗi  $v \in V$ )

Đánh dấu  $v$  chưa được thăm;

for (mỗi  $v \in V$ )

if ( $v$  chưa được thăm)

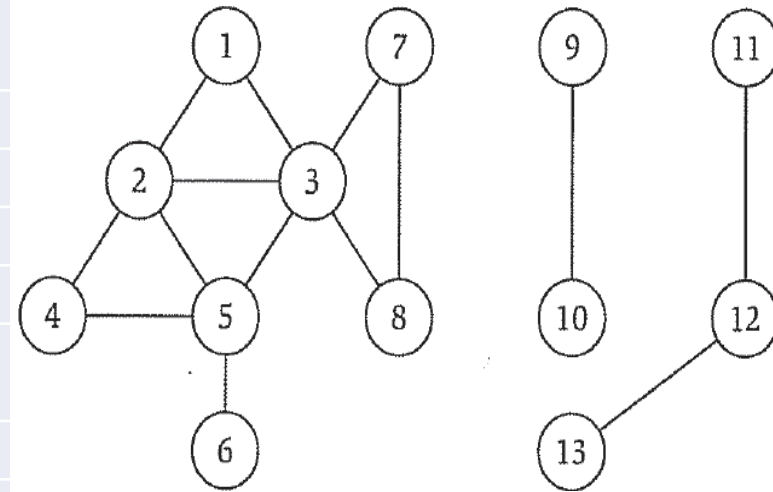
Thăm  $v$  và đánh dấu  $v$  đã được thăm;

DFS( $v$ );

- Ứng dụng
  - Phân lớp các cung
  - Phát hiện đồ thị có chu trình hay không

## 5.3. Một số bài toán tiêu biểu

| Bước | Đỉnh hiện tại | Hàng xóm        | Hành động                                           | Thứ tự duyệt đến đây                                                                                  |
|------|---------------|-----------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1    | 1             | {2, 3}          | Đi sang 2                                           | $1 \rightarrow 2$                                                                                     |
| 2    | 2             | {1, 3, 4, 5}    | 1 đã thăm $\rightarrow$ đi sang 3                   | $1 \rightarrow 2 \rightarrow 3$                                                                       |
| 3    | 3             | {1, 2, 5, 7, 8} | 1,2 đã thăm $\rightarrow$ đi sang 5                 | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$                                                         |
| 4    | 5             | {2, 3, 4, 6}    | 2,3 đã thăm $\rightarrow$ đi sang 4                 | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$                                           |
| 5    | 4             | {2, 5}          | Tất cả đã thăm $\rightarrow$ quay lui về 5          | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$                                           |
| 6    | 5             | {2, 3, 4, 6}    | Sau 4 $\rightarrow$ đi sang 6                       | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$                             |
| 7    | 6             | {5}             | Đã thăm $\rightarrow$ quay lui về 5 $\rightarrow$ 3 | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$                             |
| 8    | 3             | {1, 2, 5, 7, 8} | Sau 5 $\rightarrow$ đi sang 7                       | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7$               |
| 9    | 7             | {3, 8}          | 3 đã thăm $\rightarrow$ đi sang 8                   | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8$ |
| 10   | 8             | {3, 7}          | Đều đã thăm $\rightarrow$ quay lui, kết thúc        | $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8$ |



Kết quả DFS:  $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8$

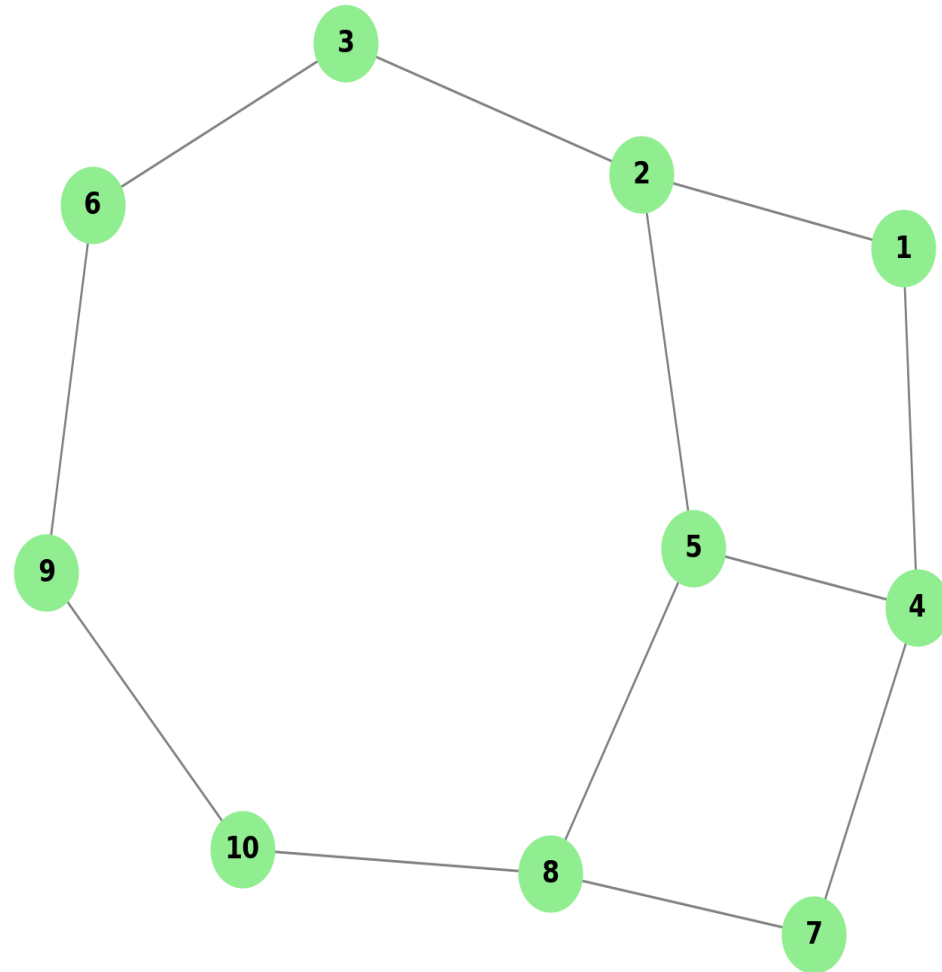
## 5.3. Một số bài toán tiêu biểu

**Thứ tự BFS (Breadth-First Search) từ đỉnh 1:**

$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 10$

**Kết quả DFS (Depth-First Search) từ đỉnh 1:**

$1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 10 \rightarrow 8 \rightarrow 5 \rightarrow 4 \rightarrow 7$



# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu

5.3.1. Đi qua đồ thị

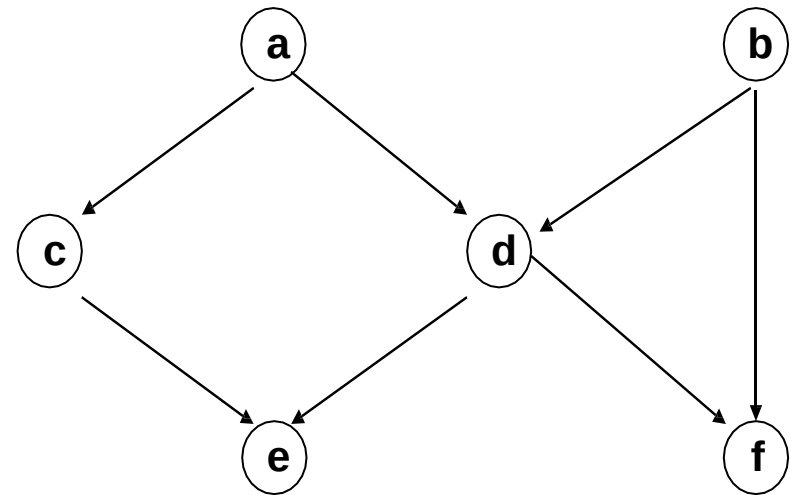
→ 5.3.2. Sắp xếp topo

5.3.3. Tìm đường đi ngắn nhất

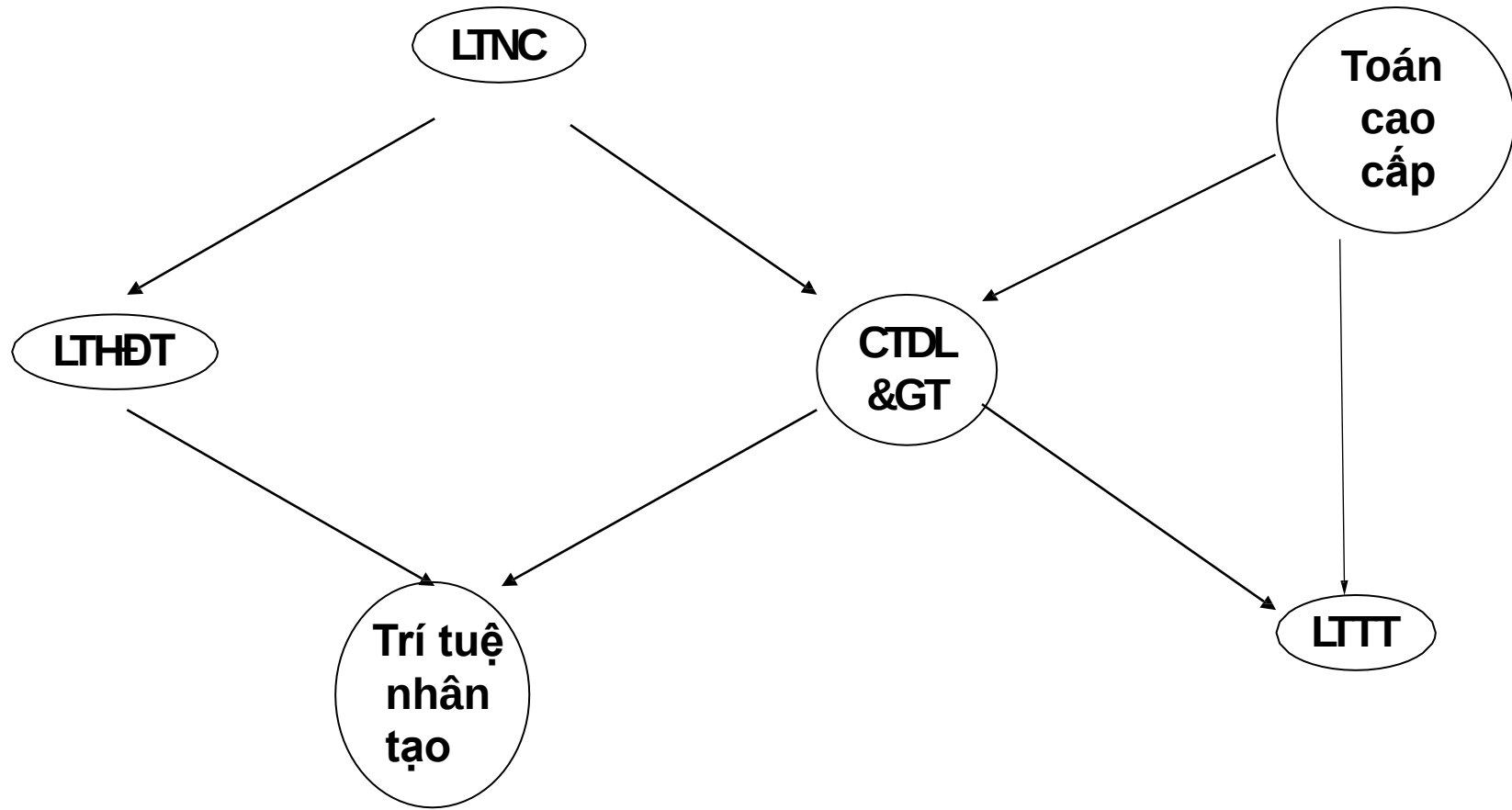
5.3.4. Tìm cây bao trùm ngắn nhất

## 5.3.2. Sắp xếp Topo

- Đồ thị định hướng không chu trình
  - Thuật ngữ
    - Directed acyclic graph (DAG)
    - Acyclic digraph
  - Nhiều dạng quan hệ trên một tập đối tượng có thể biểu diễn bởi DAG. Ví dụ:
    - Quan hệ tiên quyết giữa các môn học trong một chương trình học
    - Quan hệ thứ tự thời gian giữa các nhiệm vụ trong một đề án
    - Quan hệ thứ tự bộ phận trên một tập A

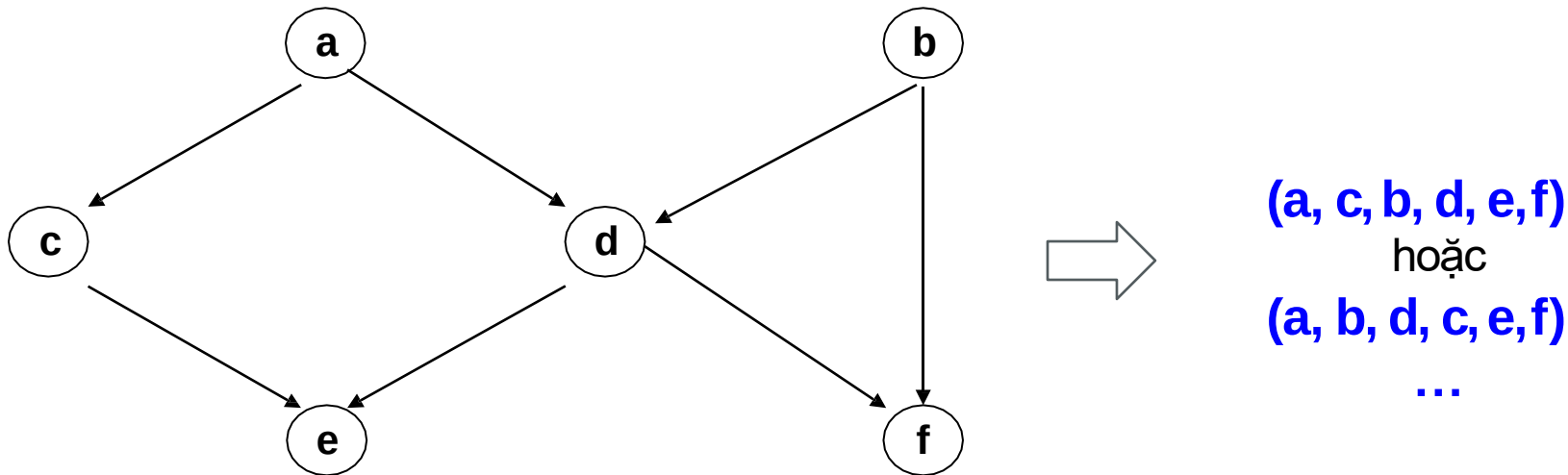


## 5.3.2. Sắp xếp Topo



## 5.3.2. Sắp xếp Topo

- Sắp xếp topo (topological sort)
  - Cho  $G = (V, E)$  là một DAG, ta cần sắp xếp các đỉnh của đồ thị thành một danh sách sao cho nếu có cung  $(u, v)$  thì  $u$  cần phải đứng trước  $v$  trong danh sách đó.



- Dùng kĩ thuật tìm kiếm theo độ sâu?

## 5.3.2. Sắp xếp Topo

- Ý tưởng topo sort dựa trên DFS

Algorithm *DFS(v)*

// Tìm kiếm theo độ sâu xuất phát từ v.

*Input:* Đỉnh v chưa được thăm for (mỗi đỉnh u kề v)

if ( u chưa được thăm)

Đánh dấu u đã được thăm;

DFS(u)

Algorithm *DFSTraversal(G)*

// Đi qua đồ thị  $G=(V, E)$  theo độ sâu

for (mỗi  $v \in V$ ) Đánh dấu v chưa được thăm;

for (mỗi  $v \in V$ )

if (v chưa được thăm)

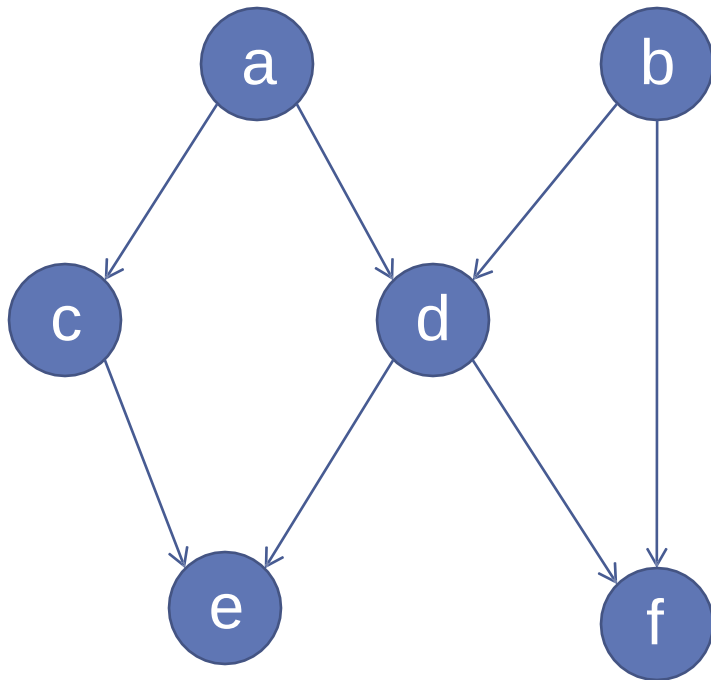
Thăm v và đánh dấu v đã được thăm; DFS(v);

- Thực hiện DFSTraversal trên đồ thị G, thêm lệnh `L.append(v)` vào cuối hàm DFS(v)
- Đảo ngược L
- *[Tác giả: Tarjan]*



## 5.3.2. Sắp xếp Topo

- Minh hoạ topo sort



DFS(a)

- DFS(c)
  - DFS(e)
    - $L = (e)$
  - $L = (e, c)$
- DFS(d)
  - DFS(f)
    - $L = (e, c, f)$
  - $L = (e, c, f, d)$
  - $L = (e, c, f, d, a)$
- DFS(b)
  - $L = (e, c, f, d, a, b)$
  - $L = (b, a, d, f, c, e)$

# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu

5.3.1. Đi qua đồ thị

5.3.2. Sắp xếp topo

→ 5.3.3. Tìm đường đi ngắn nhất

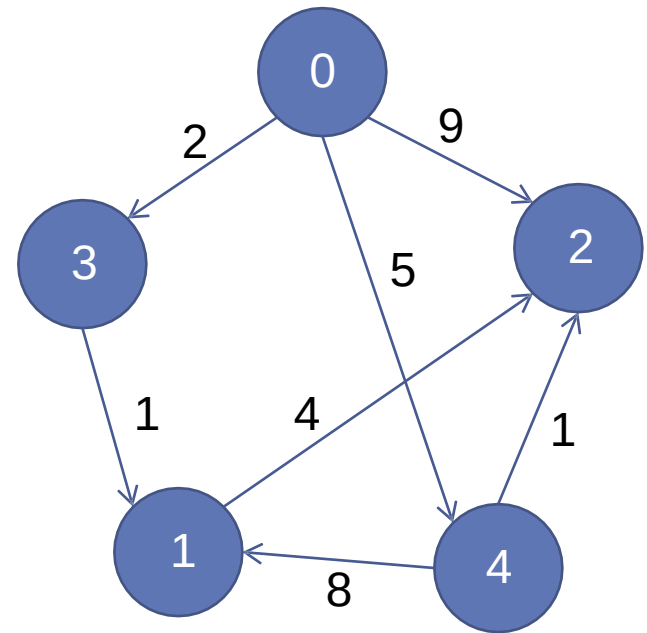
5.3.4. Tìm cây bao trùm ngắn nhất

## 5.3. Một số bài toán tiêu biểu

- Tổng quan
  - Tìm đường đi ngắn nhất trong đồ thị
    - Không trọng số: Dùng BFS
    - Có trọng số
      - Trọng số có thể âm:
        - Bellman-Ford
      - Trọng số không âm: Dijkstra
  - Xét hai vấn đề
    - Tìm đường đi ngắn nhất từ một đỉnh nguồn tới các đỉnh còn lại.
      - single-source shortest path problem
    - Tìm đường đi ngắn nhất giữa mọi cặp đỉnh của đồ thị.
      - all-pairs shortest path problem

## 5.3. Một số bài toán tiêu biểu

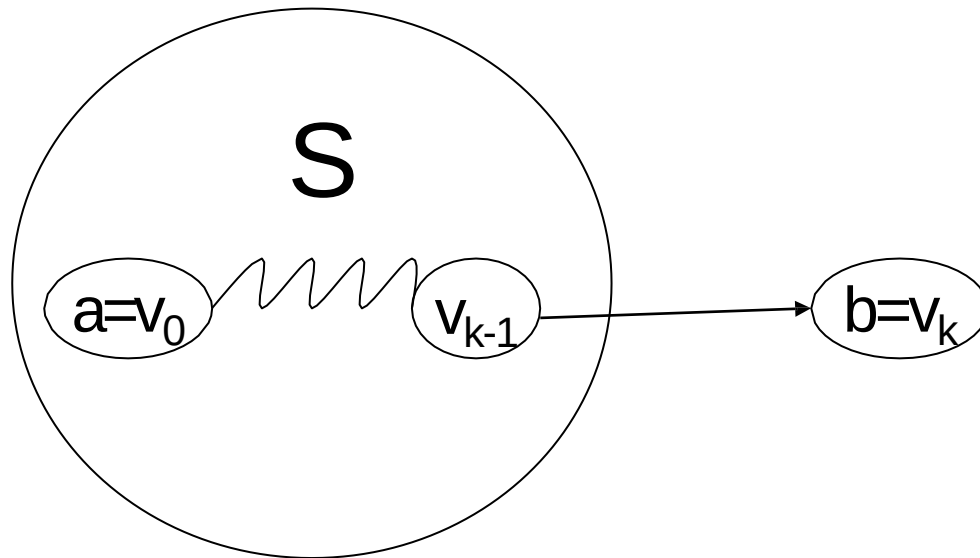
- Thuật toán Dijkstra cho bài single-source
  - Ví dụ: tìm đường đi ngắn nhất từ đỉnh nguồn là đỉnh 0
  - Thiết kế dựa vào kỹ thuật tham ăn
  - Xác định đường đi ngắn nhất từ đỉnh nguồn a tới các đỉnh còn lại qua các bước
    - Mỗi bước ta xác định đường đi ngắn nhất từ a tới một đỉnh
    - Lưu các đỉnh đã xác định đường đi ngắn nhất từ a tới chúng vào tập S
    - Ban đầu tập S chỉ chứa một đỉnh nguồn a



## 5.3. Một số bài toán tiêu biểu

- Thuật toán Dijkstra

- Gọi đường đi từ  $a$  tới đỉnh  $b$  là đường đi đặc biệt nếu đường đi đó chỉ đi qua các đỉnh trong  $S$
- Dùng mảng  $D$ : Độ dài đường đi đặc biệt từ  $a$  tới  $b$  lưu trong  $D[b]$ 
  - Ban đầu  $S = \{a\}$ ,  $D[a] = 0$ ,  $D[b] = c(a, b)$  với  $b \neq a$

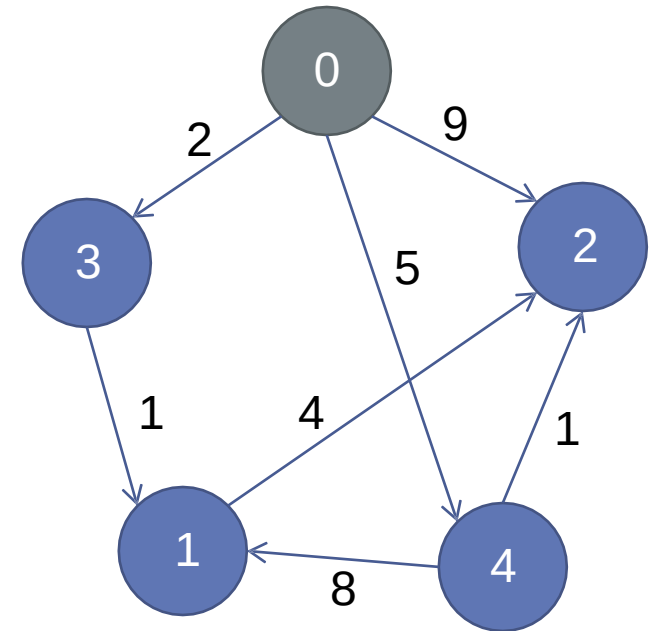


## 5.3. Một số bài toán tiêu biểu

- Thuật toán Dijkstra
  - Dùng mảng D: Độ dài đường đi đặc biệt từ a tới b lưu trong D[b]
    - Ban đầu  $S = \{a\}$ ,  $D[a] = 0$ ,  $D[b] = c(a, b)$  với  $b \neq a$
    - Tại mỗi bước
      - Chọn một đỉnh u không thuộc S mà D[u] nhỏ nhất và thêm u vào S
        - xem D[u] là độ dài đường đi ngắn nhất từ a tới u
      - Sau đó, xác định lại các D[b] với b ở ngoài S
        - $D[b] = \min(D[b], D[u] + c(u, b))$
      - Lặp lại cho tới khi S gồm tất cả các đỉnh của đồ thị

## 5.3. Một số bài toán tiêu biểu

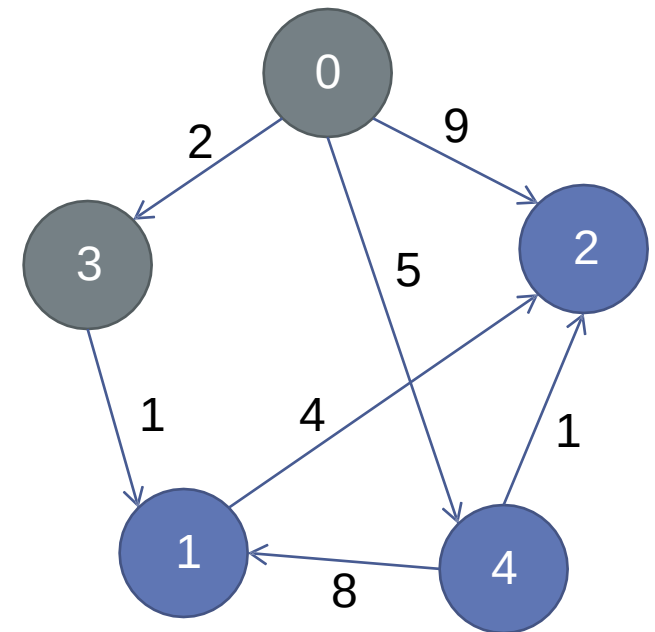
- Minh họa thuật toán Dijkstra: Ban đầu
  - $S = \{0\}$
  - $D[0] = 0$
  - $D[1] = \infty$
  - $D[2] = 9$
  - $D[3] = 2$
  - $D[4] = 5$



## 5.3. Một số bài toán tiêu biểu

- Minh họa thuật toán Dijkstra: Ban đầu

- $S = \{0, 3\}$
- $D[0] = 0$
- $D[1] = \min(\infty, D[3] + 1) = 3$
- $D[2] = \min(9, D[3] + \infty) = 9$
- $D[3] = 2$
- $D[4] = \min(5, D[3] + \infty) = 5$

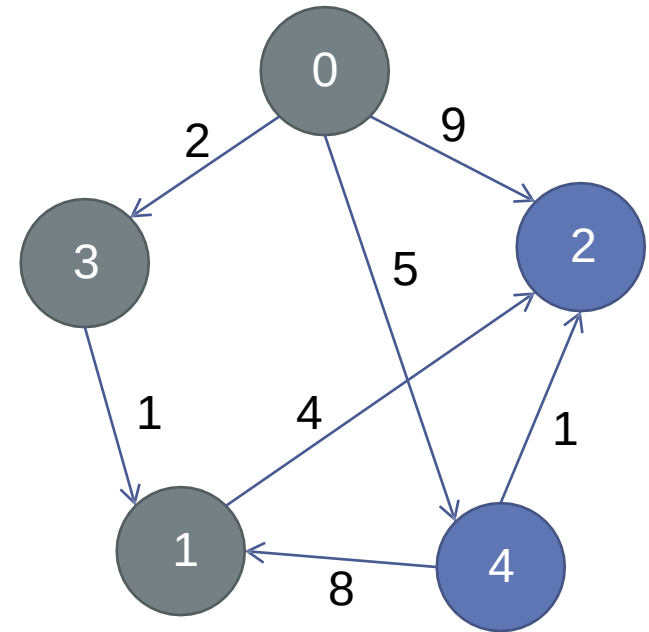




## 5.3. Một số bài toán tiêu biểu

- Minh họa thuật toán Dijkstra: Ban đầu

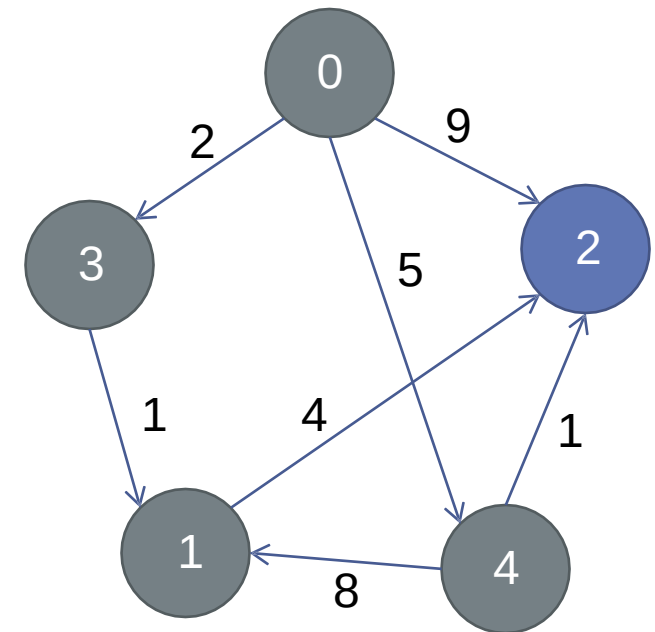
- $S = \{0, 3, 1\}$
- $D[0] = 0$
- $D[1] = 3$
- $D[2] = \min(9, D[1] + 4) = 7$
- $D[3] = 2$
- $D[4] = \min(5, D[1] + \infty) = 5$



## 5.3. Một số bài toán tiêu biểu

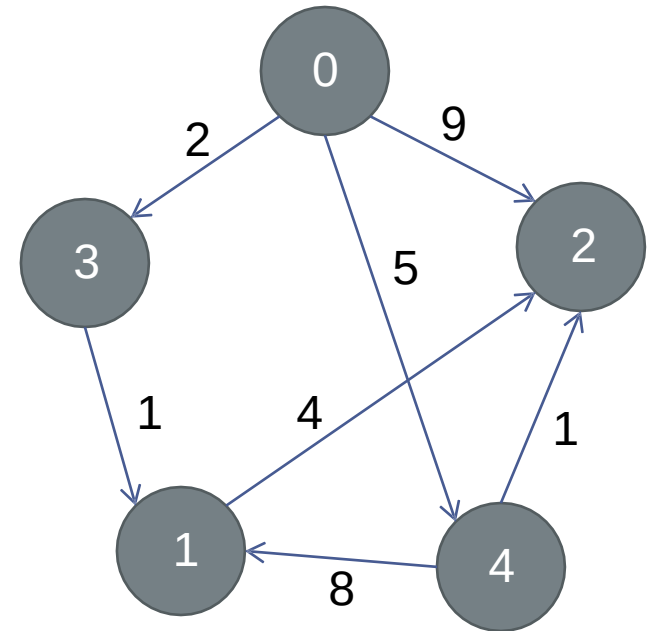
- Minh họa thuật toán Dijkstra: Ban đầu

- $S = \{0, 3, 1, 4\}$
- $D[0] = 0$
- $D[1] = 3$
- $D[2] = \min(7, D[4] + 1) = 6$
- $D[3] = 2$
- $D[4] = 5$

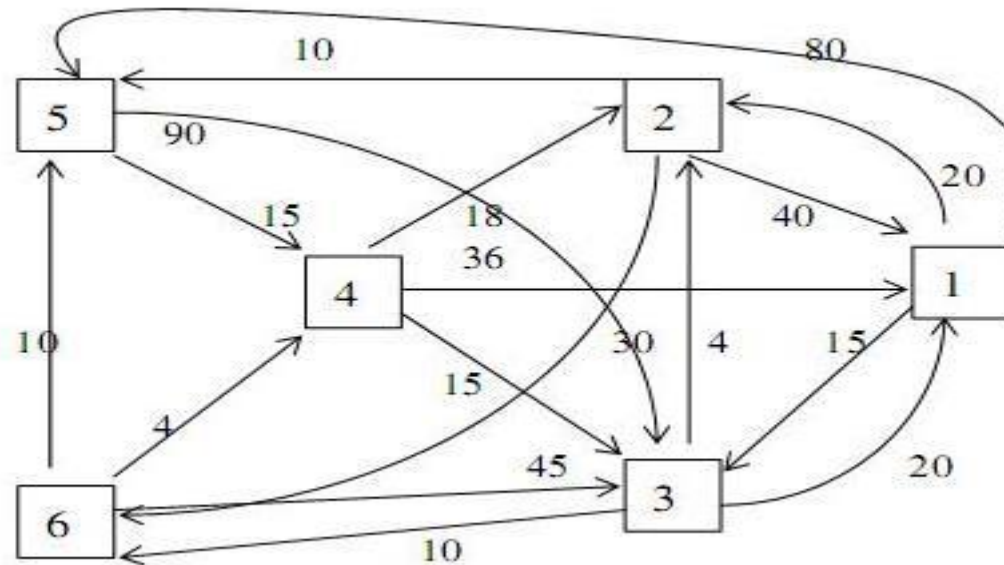


## 5.3. Một số bài toán tiêu biểu

- Minh họa thuật toán Dijkstra: Ban đầu
  - $S = \{0, 3, 1, 4, 2\}$
  - $D[0] = 0$
  - $D[1] = 3$
  - $D[2] = 6$
  - $D[3] = 2$
  - $D[4] = 5$
  - $D[b]$  lưu độ dài đường đi ngắn nhất từ  $a=0$  tới  $b$ , với mọi  $b \in V$



## 5.3. Một số bài toán tiêu biểu



| Bước<br>lập | Đường đi ngắn<br>nhất là đường đi<br>từ đỉnh 1 | đến<br>đỉnh | Chiều dài của đường đi ngắn nhất<br>từ đỉnh s (=1) đến các đỉnh khác : tsnn[] |    |    |    |    |    |
|-------------|------------------------------------------------|-------------|-------------------------------------------------------------------------------|----|----|----|----|----|
|             |                                                |             | 1                                                                             | 2  | 3  | 4  | 5  | 6  |
| Bước1       | 1→3                                            | 3           | -                                                                             | 20 | 15 | ∞  | 80 | ∞  |
| Bước2       | 1→3→2                                          | 2           | -                                                                             | 19 | -  |    |    | 25 |
| Bước3       | 1→3→6                                          | 6           | -                                                                             | -  | -  |    | 29 | 25 |
| Bước4       | 1→3→6→4                                        | 4           | -                                                                             | -  | -  | 29 |    | -  |
| Bước5       | 1→3→2→5                                        | 5           | -                                                                             | -  | -  | -  | 29 | -  |

## 5.3. Một số bài toán tiêu biểu

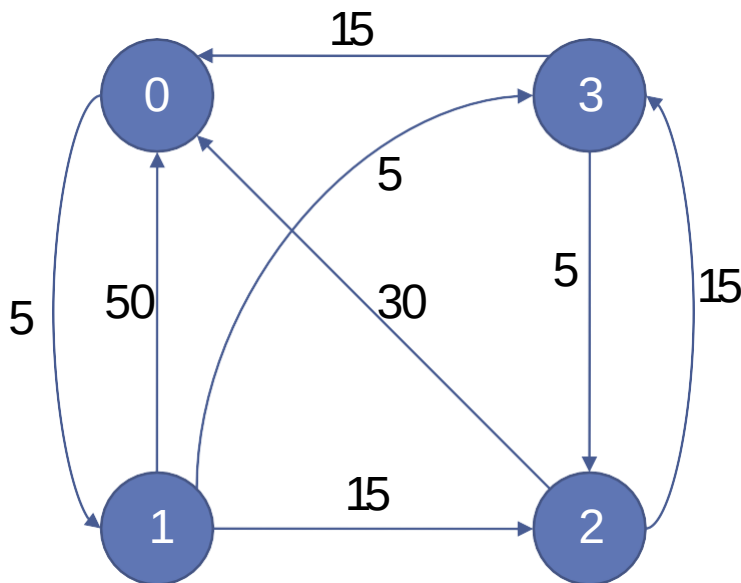
- Các vấn đề khác
  - Ghi lại vết đường đi ngắn nhất từ nguồn tới các đỉnh khác
  - Tính đúng đắn của thuật toán Dijkstra
  - Dùng hàng ưu tiên lưu tập đỉnh ngoài S để tăng hiệu quả
    - $O(|V|\log|V| + |E|\log|V|)$

## 5.3. Một số bài toán tiêu biểu

- Thuật toán Floyd cho bài all-pairs
  - Thiết kế dựa trên kỹ thuật quy hoạch động
  - Ký hiệu  $S_k$  là tập các đỉnh từ 0 đến k
    - $S_k = \{0, 1, \dots, k\}$ ,  $k \leq n-1$
  - Gọi  $A_k(i, j)$  là độ dài đường đi ngắn nhất từ đỉnh i tới đỉnh j nhưng chỉ đi qua các đỉnh trong tập  $S_k$ 
    - Khi  $k = n-1$  thì  $S_{n-1} = V \Rightarrow A_{n-1}(i, j)$  chính là đường đi ngắn nhất từ i tới j trong đồ thị đã cho
    - Khi  $k = -1$ ,  $S_k$  rỗng  $\Rightarrow A_{-1}(i, j) = c(i, j)$

## 5.3. Một số bài toán tiêu biểu

- Minh họa:  $k = -1$
- $S_{-1}$  rỗng,  $A_{-1}(i,j)$  cho trong bảng



|   | 0  | 1        | 2        | 3        |
|---|----|----------|----------|----------|
| 0 | 0  | 5        | $\infty$ | $\infty$ |
| 1 | 50 | 0        | 15       | 5        |
| 2 | 30 | $\infty$ | 0        | 15       |
| 3 | 15 | $\infty$ | 5        | 0        |

## 5.3. Một số bài toán tiêu biểu

- Công thức tính  $A_k$  từ  $A_{k-1}$ 
  - Nhận xét quan trọng
    - Nếu đỉnh  $k$  nằm trên đường đi ngắn nhất từ đỉnh  $i$  tới đỉnh  $j$  thì đoạn đường từ  $i$  tới  $k$  và đoạn đường từ  $k$  tới  $j$  phải là đường đi ngắn nhất từ  $i$  tới  $k$  và từ  $k$  tới  $j$  tương ứng.
    - Nếu  $A_k(i,j)$  là độ dài đường đi không qua đỉnh  $k$ , tức là đường đi này chỉ đi qua các đỉnh trong  $S_{k-1}$  thì

$$A_k(i,j) = A_{k-1}(i,j)$$

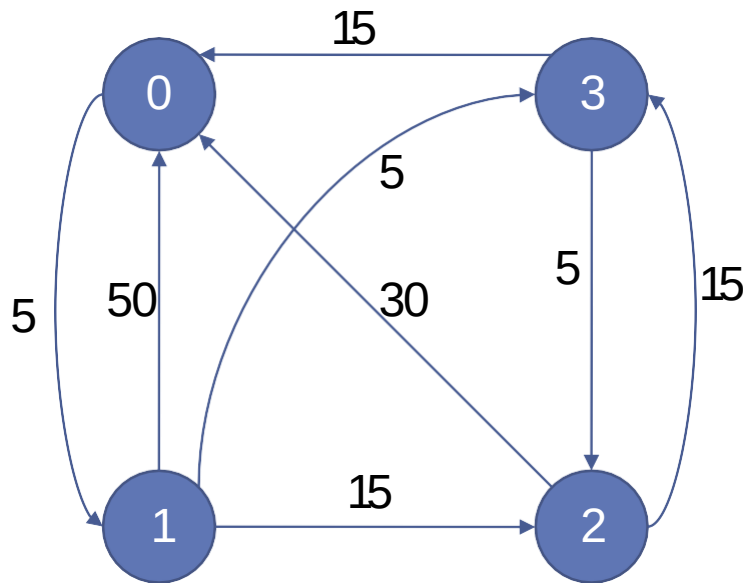
- Nếu  $A_k(i,j)$  là độ dài của đường đi qua đỉnh  $k$  thì trên đường đi này đoạn từ  $i$  tới  $k$  có độ dài là  $A_{k-1}(i,k)$ , còn đoạn đường từ  $k$  tới  $j$  có độ dài là  $A_{k-1}(k,j)$
- Do đó

$$A_k(i,j) = \min( A_{k-1}(i,j) , A_{k-1}(i,k) + A_{k-1}(k,j) )$$



## 5.3. Một số bài toán tiêu biểu

- Minh họa:  $k = 0 \dots ?$



$A_1$

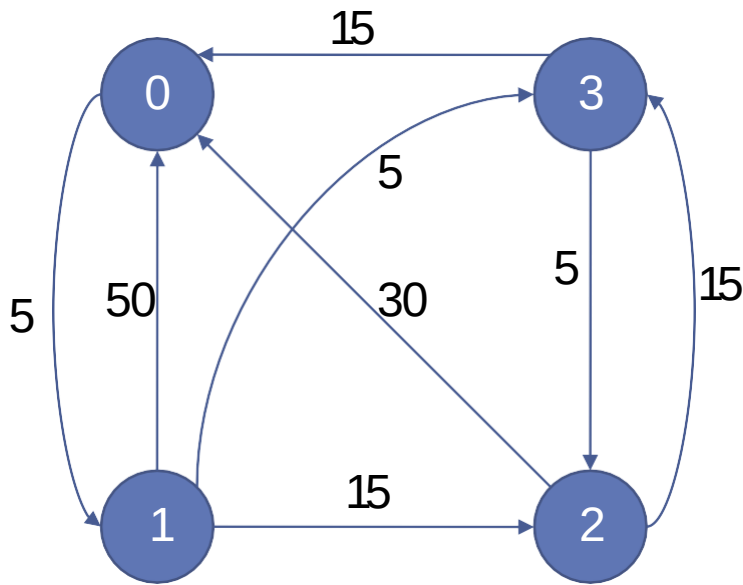
|   | 0  | 1        | 2        | 3        |
|---|----|----------|----------|----------|
| 0 | 0  | 5        | $\infty$ | $\infty$ |
| 1 | 50 | 0        | 15       | 5        |
| 2 | 30 | $\infty$ | 0        | 15       |
| 3 | 15 | $\infty$ | 5        | 0        |

$A_0$

|   | 0         | 1        | 2        | 3        |
|---|-----------|----------|----------|----------|
| 0 | <b>0</b>  | <b>5</b> | $\infty$ | $\infty$ |
| 1 | <b>50</b> | 0        | 15       | 5        |
| 2 | <b>30</b> | 35       | 0        | 15       |
| 3 | <b>15</b> | 20       | 5        | 0        |

## 5.3. Một số bài toán tiêu biểu

- Minh họa:  $k = 0 \dots ?$



$A_0$

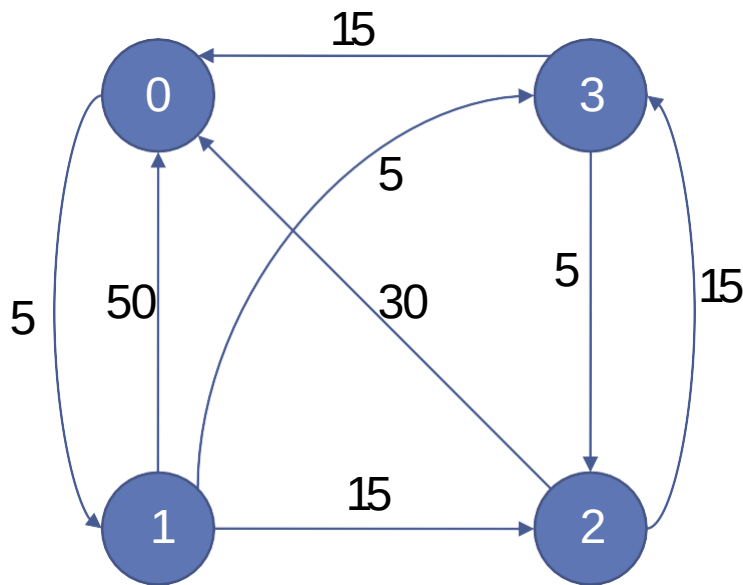
|   | 0  | 1  | 2        | 3        |
|---|----|----|----------|----------|
| 0 | 0  | 5  | $\infty$ | $\infty$ |
| 1 | 50 | 0  | 15       | 5        |
| 2 | 30 | 35 | 0        | 15       |
| 3 | 15 | 20 | 5        | 0        |

$A_1$

|   | 0         | 1         | 2         | 3        |
|---|-----------|-----------|-----------|----------|
| 0 | 0         | <b>5</b>  | 20        | 10       |
| 1 | <b>50</b> | 0         | <b>15</b> | <b>5</b> |
| 2 | 30        | <b>35</b> | 0         | 15       |
| 3 | 15        | <b>20</b> | 5         | 0        |

## 5.3. Một số bài toán tiêu biểu

- Minh họa:  $k = 0 \dots ?$



$A_1$

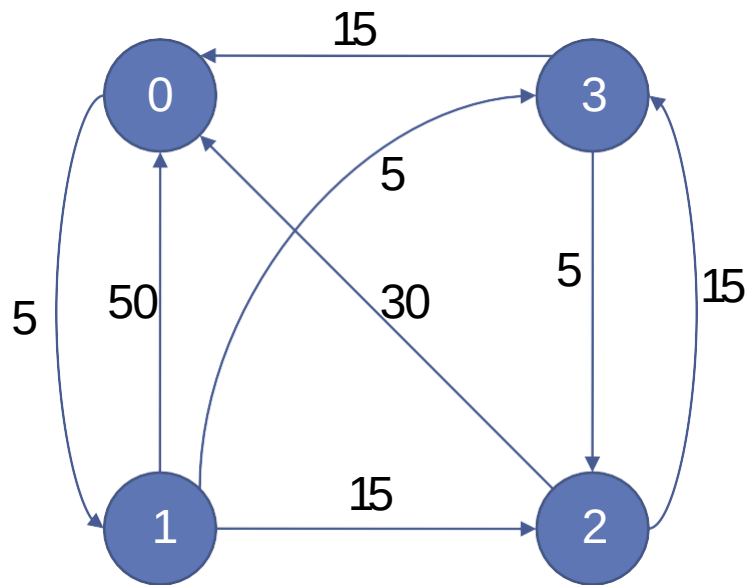
|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 0  | 5  | 20 | 10 |
| 1 | 50 | 0  | 15 | 5  |
| 2 | 30 | 35 | 0  | 15 |
| 3 | 15 | 20 | 5  | 0  |

$A_2$

|   | 0         | 1         | 2         | 3         |
|---|-----------|-----------|-----------|-----------|
| 0 | 0         | 5         | <b>20</b> | 10        |
| 1 | 45        | 0         | <b>15</b> | 5         |
| 2 | <b>30</b> | <b>35</b> | 0         | <b>15</b> |
| 3 | 15        | 20        | <b>5</b>  | 0         |

## 5.3. Một số bài toán tiêu biểu

- Minh họa:  $k = 0 \dots ?$



$A_2$

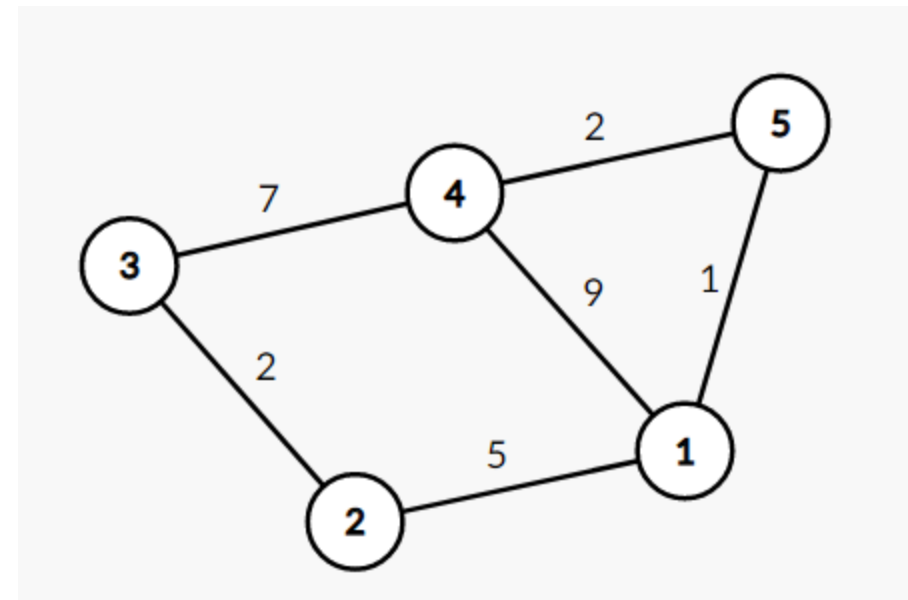
|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 0  | 5  | 20 | 10 |
| 1 | 45 | 0  | 15 | 5  |
| 2 | 30 | 35 | 0  | 15 |
| 3 | 15 | 20 | 5  | 0  |

$A_3$

|   | 0         | 1         | 2         | 3  |
|---|-----------|-----------|-----------|----|
| 0 | 0         | 5         | <b>15</b> | 10 |
| 1 | 20        | 0         | <b>10</b> | 5  |
| 2 | 30        | 35        | 0         | 15 |
| 3 | <b>15</b> | <b>20</b> | <b>5</b>  | 0  |

## 5.3. Một số bài toán tiêu biểu

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 3 | 1 |
| 2 | 5 | 0 | 2 | 8 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 3 | 8 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |



# Chương 5: Đồ thị

5.1. Đồ thị và các khái niệm liên quan

5.2. Cài đặt đồ thị

5.3. Một số bài toán tiêu biểu

5.3.1. Đi qua đồ thị

5.3.2. Sắp xếp topo

5.3.3. Tìm đường đi ngắn nhất

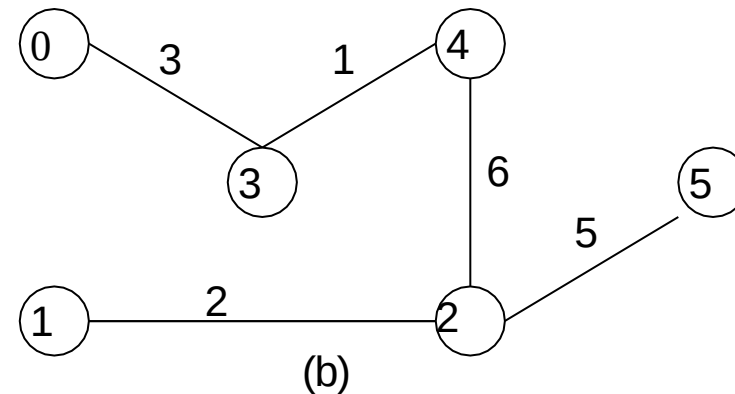
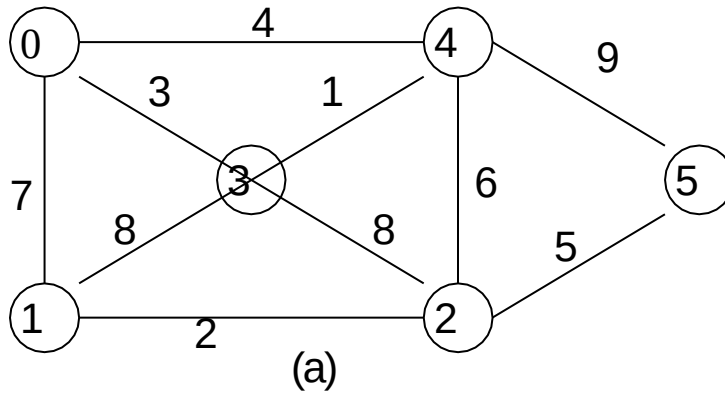
→ 5.3.4. Tìm cây bao trùm ngắn nhất

## 5.3. Một số bài toán tiêu biểu

- Bài toán
  - $G = (V, E)$  là đồ thị vô hướng liên thông
  - $G' = (V, T)$  có  $T \subseteq E$ , liên thông và không có chu trình được gọi là cây bao trùm của  $G$ 
    - Cây này có  $|V| - 1$  cạnh
  - Ta cần tìm cây bao trùm ngắn nhất của một đồ thị  $G$  vô hướng liên thông có trọng số không âm
    - Tức là cây bao trùm có tổng độ dài các cạnh là nhỏ nhất
    - Thuật ngữ: minimum spanning tree (MST)

## 5.3. Một số bài toán tiêu biểu

- Minh họa cây bao trùm ngắn nhất



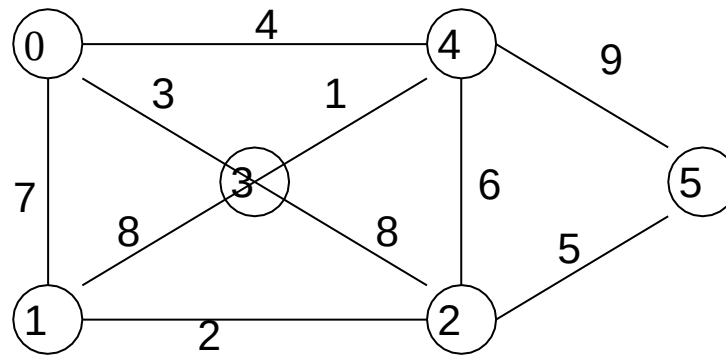


## 5.3. Một số bài toán tiêu biểu

- Ý tưởng
  - Thiết kế theo kỹ thuật tham ăn
  - Xây dựng tập  $T$  các cạnh dần từng bước xuất phát từ  $T$  rỗng
  - Trong mỗi bước lặp, ta sẽ chọn cạnh  $(u,v)$  ngắn nhất trong các cạnh còn lại để đưa vào tập  $T$ 
    - Prim:  $T \cup (u, v)$  phải liên thông, không có chu trình
    - Kruskal:  $T \cup (u, v)$  không có chu trình
      - Sử dụng KDLTTHọ các tập con không cắt nhau (disjoint set ADT) [chương 13]

## 5.3. Một số bài toán tiêu biểu

- Minh hoạ



## 5.3. Một số bài toán tiêu biểu

- Các vấn đề khác
  - Độ phức tạp thời gian
  - Tính đúng đắn